Name Sort: Bucketing Person Names by Character Frequency, and When MSD Radix Sort Is Enough

Attila Torda

October 2026

Abstract

Person names are far from uniformly distributed: in the 2010 US Census, 9.6 % of people have a surname beginning with M and 0.04 % one beginning with X, and single prefixes such as *Ma* or *Wil* hold more than one percent each. We ask whether a sort can profit from knowing this in advance. *Name sort* is a bucket sort whose buckets are cut from character-frequency tables, one per position for the first three characters of a name, built from public Census data and refined on a small sample of the input. It cuts a trie of at most three levels so that each leaf is expected to hold about 32 names, packs the leaves into buckets, distributes the names in two linear passes and finishes each bucket with insertion sort or MSD radix sort. On a million synthetic "Surname Firstname" names it is 3.32× faster than `std::sort` and 1.36× faster than a plain MSD radix sort (GCC; MSVC: 3.27× and 1.75×). The evaluation is mostly a negative-leaning one, though. On English names the frequency tables add nothing measurable over the sample alone; deeper tries that cut more even buckets are slower; the plan pays only from about 500,000 synthetic names; and on the 162,253 real Census surnames plain MSD radix sort is faster. We report this together with sorts that the standard libraries of C, C++, Java, Python, Rust, Go, C# and JavaScript do not offer: an LSD radix sort (up to 3.48× faster than `std::sort` on 16 Mi keys), a sampled bucket sort, an AVX2 bitonic sort for any n, a bitonic merge sort with an AVX2 merge network (3.40×), and a GPU bitonic sort through OpenCL (5.15×, copies included), all in an open-source C library with every number in this paper regenerated from its benchmark files.

1 Introduction

Sorting names is among the most common jobs of everyday software: contact lists, member and patient registers, class lists, mailing lists. Library sorts treat names as opaque strings and compare them, $\Theta(n \log n)$ times, each comparison reading bytes that the previous ones have mostly read already. String sorts that inspect characters instead—MSD radix sort [10, 13], multikey quicksort [3], burstsort [26]—avoid most of this, but they learn the distribution of the characters as they go.

Names, however, have a distribution that is known before the input arrives. Public statistics list how many people carry each surname and first name [28, 29], and the distribution is very uneven (Section 3). A sort that knows where the dense regions are can cut buckets of equal expected size in one step, the idea behind samplesort [1, 7, 24] and learned sorting [12], but with the model fitted on millions of real names instead of the input.

This paper reports what that buys. Our contributions are:

- **Name sort** (Section 4): a bucket sort whose bucket boundaries come from per-position character-frequency tables of person names, blended with a sample of the input. A small trie of lookup tables distributes the names, and every bucket is finished with a comparison sort

Language	Library sort	Algorithm
C	`qsort`	unspecified (usually introsort or merge sort)
C++	`std::sort`, `std::stable_sort`	introsort [17], merge sort
Java	`Arrays.sort`	dual-pivot quicksort (primitives), TimSort (objects) [20]
Python	`list.sort`	TimSort [22] with the Powersort merge policy [16]
Rust	`sort`, `sort_unstable`	driftsort, ipnsort [4]
Go	`slices.Sort`	pattern-defeating quicksort [21, 27]
C#	`Array.Sort`	introsort [15]
JavaScript	`Array.prototype.sort`	TimSort (V8) [30]

Table 1: Standard-library sorts. None offers a radix, bucket or bitonic sort; Java uses counting sort for `byte`, `short` and `char` arrays.

or MSD radix sort that skips the bytes its prefix fixes. The order is byte order (UTF-8 code point order) or ASCII-caseless.

- **An evaluation that includes what did not work** (Section 7): synthetic and real name lists, two compilers, and an ablation of every design choice against a plain MSD radix sort measured in the same interleaved runs.
- **Sorts the standard libraries lack** (Section 5): LSD radix sort, a sampled bucket sort and a bitonic sort for any n with an AVX2 kernel, for integers and floating-point numbers in IEEE 754 total order; and, built on the bitonic network, a bitonic merge sort, a bitonic bucket sort and a GPU bitonic sort (OpenCL) that needs no SDK at build time.
- **Reproducibility**: the library, the tables' generator, the harness and the scripts that produce every number, table and figure of this paper are public (Section 9).

2 Background

What standard libraries offer. Every mainstream standard library sorts by comparison only (Table 1). Radix sort appears in add-on libraries such as Boost.Sort [23] and in GPU libraries [14, 25]. A radix-sort path for primitive arrays in Java's dual-pivot quicksort has been under review since 2021 [19]; we could not confirm that it has been released.

String sorting. MSD radix sort distributes strings by their first byte into 256 buckets (plus one for strings that end), then recurses on each bucket with the next byte [11, 13]. It reads each byte that distinguishes a string from its neighbours about once, but every pass pays for a 257-entry count, which dominates on small buckets; implementations therefore switch to insertion sort or multikey quicksort [3] below a few dozen strings, and cache-conscious variants keep the counts and the moved data small [10]. Burstsort [26] builds a trie of string prefixes whose leaves are buckets that "burst" into subtries when they overflow.

Distribution sorts with a model. Samplesort [7] draws a sample, sorts it and uses evenly spaced sample elements as bucket boundaries; super scalar samplesort [24] and IPS^{4}o [1] make the classification branch-free and in-place. Learned sort [12] replaces the sample's splitters by a model of the key distribution trained on the sample. Name sort is a samplesort whose model is a prefix trie with a prior from outside the input.

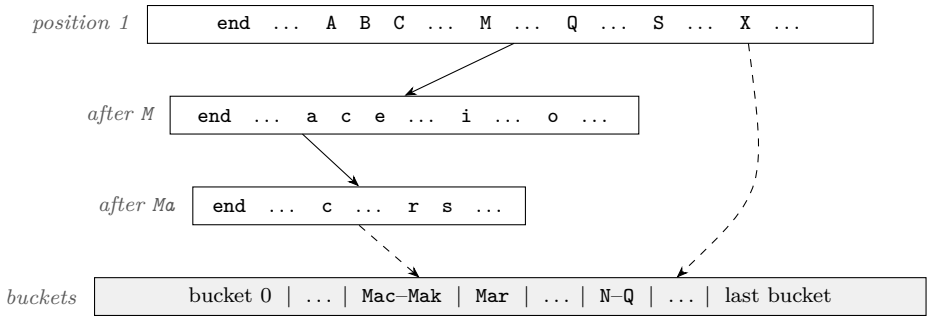


Figure 1: A name-sort plan (schematic). Each block is a 257-entry lookup table for one position. Heavy symbols point to a child block for the next position; light ones point to a leaf, and runs of light symbols share one leaf. The leaves, in order, are packed into buckets of about 32 expected names. A name is classified with at most three lookups.

3 The distribution of names

We use the 2010 US Census list of 162,253 surnames, each held by at least 100 people and covering about 266 million people [29], and the 1990 Census lists of 4,275 female and 1,219 male first names [28]. Both are in the public domain.

The first letter alone is very uneven: weighted by the people who carry them, surnames begin with M (9.6 %), S (9.4 %), B (8.5 %), C (7.7 %) and H (7.1 %) far more often than with Q (0.24 %), U (0.23 %) or X (0.04 %). Equal-width buckets on the first letter would range from nearly empty to one tenth of the input. Some prefixes are heavy on their own: *Ma* (3.0 %), *Ha* (2.5 %), *Mc* (1.7 %), and among three-letter prefixes *Wil* (1.3 %), *Mar* (1.2 %), *Smi* (0.95 %), *Joh* (0.83 %) and *Sch* (0.78 %). Splitting the heavy first letters on the second and third letter is what makes buckets of equal expected size possible.

The tables. A generator script downloads the lists and computes four tables over 257 symbols: symbol 0 is the end of a name, symbol $1 + b$ is the byte b. Table P_k, for $k = 1, 2, 3$, gives the share of each symbol at position k among the names that reach position k (a name of length $k - 1$ contributes the end symbol); table P_* counts the symbols at every position, with one end symbol per name. The names are weighted by the number of people who carry them, written in title case as people type them (“Smith”), and mixed half surnames, half first names, so that lists of either kind and lists of “Surname Firstname” are covered. The tables are stored in parts per million in a C header (four tables of 257 entries, 4 KiB); the generator produces the same bytes on every run.

4 Name sort

Name sort sorts an array of n byte strings (pointer and length) in place. It has three phases: plan, distribute, finish (Figure 1).

4.1 Plan

Sample. Name sort takes $s = n/128$ names (at least 256, at most 16,384) at even strides and sorts them, with the same MSD radix sort used in the finishing phase.

Estimates. Let p be a prefix of d symbols, expected to hold the share $\mu(p)$ of the input, and let the sample hold s_p names with prefix p, of which s_{pc} continue with symbol c. Name sort estimates

$$\mu(pc) = \mu(p) \cdot \frac{s_{pc} + \alpha\, \pi_{d+1}(c)}{s_p + \alpha}, \tag{1}$$

where π_{d+1} is the prior for position $d+1$: P_{d+1} mixed $99:1$ with P_*, so that no byte has probability zero, and $\alpha = 16$ is the weight of the prior in sampled names. With many sampled names the sample dominates; for prefixes the sample barely sees, the Census tables do. For the ASCII-caseless order the priors of A–Z are folded into a–z.

Trie. The root block is a lookup table with 257 entries, one per symbol at position 1. A symbol c whose child is expected to hold more than $t = 32$ names ($n\,\mu(pc) > t$), and which the sample has seen at least once, gets a child block for the next position, recursively, down to depth three. Every other entry points to a leaf, and a light symbol joins the leaf of its left neighbour while the two together stay below t expected names, so a block needs only a few leaves. The number of blocks is capped at 1,024 (1 MiB of tables). The leaves, in symbol order, are then packed greedily into buckets of about t expected names, and each bucket records the number of leading bytes that all its names share: the common prefix of its first and last key.

4.2 Distribute

One pass walks the trie for every name, at most three table lookups on its first three bytes, stores the bucket id and counts the names per bucket; a prefix sum and a second pass scatter the (pointer, length) records into a buffer. Both passes are linear and read each name's first bytes only.

4.3 Finish

Buckets of up to 32 names are sorted by introsort with insertion sort for small ranges [17]; larger buckets by MSD radix sort with an explicit stack, falling back to introsort below 32 names. Both start comparing after the bucket's shared prefix. A bucket the plan misjudged, such as one holding every *Smith* of a long list, thus costs time linear in the bytes that distinguish its names, not quadratic.

Below $n_0 = 131{,}072$ names name sort does not plan at all and runs its MSD radix sort on the whole input; Section 7.2 shows why.

4.4 Order and correctness

The default order is byte order, `memcmp` with the shorter name first on a tie, which for UTF-8 is code point order. With the caseless flag, A–Z compare as a–z and names equal up to case are ordered by their bytes. Both are total orders.

Lemma 1. *If name a precedes name b in the sort order, then a's bucket is not after b's, and all names in a bucket agree on the bucket's shared prefix.*

Proof sketch. The trie classifies a name by its first (at most three) symbols, folded in the caseless order, and its blocks are indexed by symbol, so the leaf order is the order of those prefixes, with the end symbol first. Packing preserves leaf order, so the bucket id is a monotone function of the name, which proves the first claim. A bucket covers a contiguous range of keys between its first leaf's lowest and its last leaf's highest prefix, so every name in it starts with the common prefix

of those two. Names equal up to case have the same folded symbols and land in the same bucket, where the byte tie-break orders them. □

4.5 Cost

Planning sorts $s \le n/128$ names and fills at most 1,024 blocks of 257 entries; distribution is two linear passes. The finishing cost is that of the bucket sorts: at most a 32×32 insertion sort per small bucket, and for a large bucket an MSD radix sort, linear in the distinguishing bytes plus a 257-entry count per pass. Memory beyond the input is one record and one bucket id per name ($n \times 20$ bytes on 64-bit systems) plus the tables.

4.6 Unicode and collation

Because name sort works on bytes, any UTF-8 input sorts correctly in code point order; a multi-byte character simply takes more trie positions, and the sample supplies the statistics the ASCII-only Census tables lack. Language-specific order (Hungarian *cs* after *c*, German ä with a) is a collation question [6]: the caller turns each name into a collation key, a byte string whose byte order is the locale's order, and name-sorts the keys. The tables then no longer apply and the sample does all the work; Section 7.3 measures that situation on multilingual input.

5 Sorts the standard libraries lack

All three sort 32- and 64-bit unsigned and signed integers and floating-point numbers, by sorting unsigned keys: signed integers flip the sign bit, floating-point numbers flip the sign bit of positive values and all bits of negative ones. The result is IEEE 754 totalOrder [9] ($-\mathrm{NaN} < -\infty < \cdots < -0 < +0 < \cdots < +\infty < +\mathrm{NaN}$); bits are moved, never changed, so NaN payloads survive. Every function takes an optional scratch buffer and falls back to an in-place introsort if an allocation fails.

LSD radix sort. Eight-bit digits. One read pass encodes the keys and builds the histograms of all digits at once; a digit on which all keys agree is skipped, so small values, few distinct values and shared high bytes cost fewer passes [11]. Keys move between the array and the buffer once per remaining digit, and the last copy back decodes. Below 256 keys it falls back to introsort.

Bucket sort. Classic bucket sort cuts $[\min, \max]$ into equal-width buckets and works for uniform data only. Ours sorts a strided sample of 1,024 keys, takes 64 equal-depth splitters from it, and cuts every segment between two splitters into equal-width buckets of about 32 expected keys. A branch-free binary search finds the segment, a multiplication the bucket; the map is monotone, so after a counting pass, a scatter pass and an introsort per bucket the data is sorted, and a misjudged bucket only costs a larger introsort.

Bitonic sort for any n. Batcher's bitonic network [2] sorts 2^k keys with $O(n \log^2 n)$ compare-exchanges in a fixed pattern, which suits SIMD units and GPUs [5, 25]. We use the formulation in which every comparator puts the smaller key at the lower index: for each block size $k = 2, 4, \ldots$ a *flip* stage compares i with $i \oplus (k-1)$, and half-cleaner stages compare i with $i + j$ for $j = k/4, \ldots, 1$. Picture the array padded to the next power of two with $+\infty$: a comparator whose upper index lies in the padding compares against $+\infty$ and never moves anything, so it can be skipped, and the network sorts any n exactly, without padding memory. The AVX2 kernel, selected at run time, compares contiguous ranges of 8 (32-bit) or 4 (64-bit) keys with vector min/max, reverses the mirrored half of

a flip stage in a register, and runs all distances below one vector width in registers between one load and one store. AVX2 has no unsigned 64-bit comparison; we compare with the sign bits flipped.

6 Method

Machine and compilers. An AMD Ryzen 5 2600X (Zen+, 6 cores, 32 KiB L1d and 512 KiB L2 per core, 2×8 MiB L3) with 16 GB of DDR4 under Windows 10 (build 19045), the machine of our earlier studies. MinGW-w64 GCC 13.2 and Microsoft Visual C++ 19.41 (Visual Studio 2022), CMake Release builds (`-O3` and `/O2`); the AVX2 kernels alone are compiled with `-mavx2` or `/arch:AVX2`. The GPU is an NVIDIA GeForce GTX 1660 SUPER (6 GB, driver 576.02) on PCIe 3.0, used through OpenCL; the library loads the OpenCL runtime when it first needs it, and the timings include copying the keys to the GPU and back.

Timing. For each input, every algorithm first runs once untimed and its result is compared with `std::sort`'s. Then the timed runs of all algorithms are interleaved round-robin, so that slow periods of the machine affect them alike [8, 18]; each run sorts a fresh copy of the input, and the copy is not timed. We report the median of 7 runs (9 in the ablation) in nanoseconds per element, with the interquartile range in the figures. The thread is pinned to one core and runs at high priority. Separate runs of the same configuration differed by up to about 15 % in exploratory runs, so we compare algorithms only within one run, and the ablation measures the MSD radix baseline in every run.

Baselines. `std::sort` (introsort), and for names a plain MSD radix sort on string views: 257 buckets per level and insertion sort below 32 names, without plan or sample. For numbers also `std::stable_sort` and `qsort`.

Name lists. *Synthetic*: "Surname Firstname" pairs, each part drawn independently by its Census frequency, 10^6 names; the sizes below 10^6 are prefixes of the list. Its names are real, its pairs and its frequencies are realistic, but it is drawn from the data the tables were built from, which favours the tables. *Census*: the 162,253 Census surnames themselves, each once, in five orders: shuffled, popularity order (as published), sorted, reversed, and in upper case as the Census writes them. *Multilingual*: 10^6 file-name-like strings built from 47 words in 15 scripts (Czech, German, Polish, Turkish, Greek, Cyrillic, Chinese, Japanese, Korean, Arabic, Hebrew, Thai and others), 79 % of them with non-ASCII characters; it tests the byte-level path where the tables know nothing.

Numeric inputs. 2^{10} to 2^{24} keys: uniform, skewed (log-normal, $\mu = 10$, $\sigma = 2.5$), sorted, and few-unique (16 values). Bitonic sort is measured up to 2^{22}.

Correctness. Every timed result matched `std::sort` (all numeric results with GCC, all with MSVC). Besides, the unit tests compare every sort with `qsort` bit for bit on all sizes up to 1,025 and on special floating-point values, and name sort in both orders on adversarial inputs (shared prefixes of 300 bytes, embedded NUL bytes, empty names, UTF-8); every real list was also sorted in the caseless order and checked (all correct).

	GCC				MSVC			
Names	`std::sort`	MSD radix	Name sort	vs. `std::sort`	`std::sort`	MSD radix	Name sort	vs. `std::sort`
1,000	74.7	43.3	46.5	1.61×	97.1	52.4	49.5	1.96×
10,000	106	57.4	58.8	1.81×	138	69.2	62.8	2.20×
100,000	160	76.1	90.8	1.76×	193	95.9	90.4	2.14×
200,000	234	100	105	2.22×	247	127	104	2.37×
500,000	320	125	113	2.83×	337	176	114	2.96×
1,000,000	375	153	113	3.32×	393	211	120	3.27×

Table 2: Synthetic "Surname Firstname" lists: ns per name, median of 7. Below 131,072 names, name sort is its MSD radix sort without a plan.

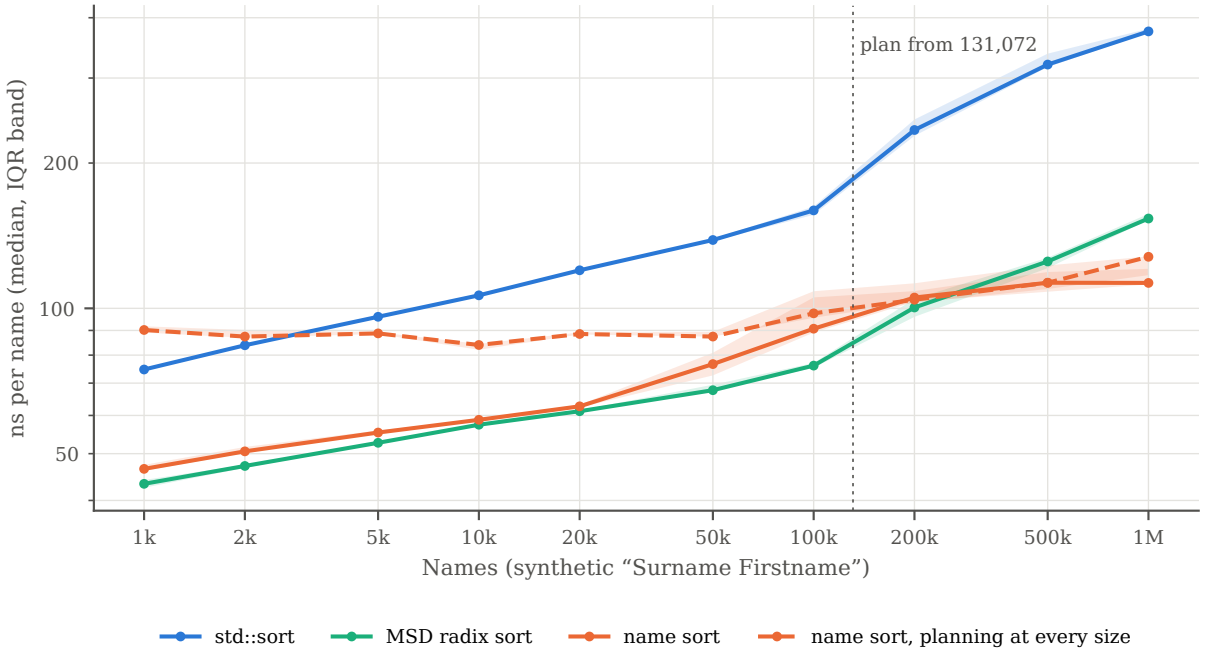


Figure 2: Synthetic names, GCC. The dashed line is name sort planning at every size (ablation run; its own MSD baseline is in Table 3); the library plans from 131,072 names on.

7 Evaluation

7.1 Synthetic names

Table 2 and Figure 2 show the synthetic lists. At 10^6 names, name sort takes 113 ns per name, against 375 for `std::sort` and 153 for plain MSD radix sort (GCC); with MSVC 120, 393 and 211. The plan cut 3,324 buckets with a mean of 301 names; the largest held 13,314 names (the common surnames, which share their first three letters), the standard deviation of the bucket sizes was 829, and MSD radix sort finished those buckets. Below the planning threshold name sort is an MSD radix sort; at 10^4 names it is 1.81× faster than `std::sort` (MSVC: 2.20×).

7.2 Ablation

Table 3 changes one choice at a time.

Configuration	Synthetic		Census		Multilingual	
	ns/name	vs. MSD	ns/name	vs. MSD	ns/name	vs. MSD
Library choices (depth 3, sample $n/128$, prior weight 16)	118	1.34×	81.1	0.80×	136	1.42×
Trie depth 2	122	1.24×	75.2	0.86×	150	1.27×
Trie depth 4	132	1.23×	99.6	0.66×	141	1.37×
Trie depth 8 (4,096 blocks)	141	1.12×	100	0.63×	132	1.43×
Prior weight 0 (sample only)	124	1.23×	76.2	0.81×	158	1.18×
Prior weight 10^9 (tables only)	117	1.38×	79.2	0.83×	177	1.08×
Sample $n/16$	124	1.27×	83.0	0.77×	150	1.27×
Introsort for buckets up to 256	127	1.20×	112	0.59×	134	1.45×

Table 3: Name sort with one design choice changed at a time, at the full list size (10^6 synthetic, 162,253 Census shuffled, 10^6 multilingual names), GCC, median of 9. "vs. MSD": plain MSD radix sort's time divided by name sort's in the same run (above 1: name sort is faster).

Depth. A deeper trie cuts more and more even buckets: at 10^6 synthetic names, depth 8 cuts 14,721 buckets whose largest holds 55,750 names, against 3,324 and 13,314 at depth 3. But it is slower: 132 (depth 4) and 141 (depth 8) against 118 ns per name. Each level is another lookup in a 1 KiB block, and with thousands of blocks those lookups miss the cache; an MSD radix sort of a large bucket is cheaper than the deeper trie that would have avoided it. The Census surnames show the same (100 against 81.1 ns); on the multilingual list, whose few distinct words leave little to split, depth makes little difference. Two levels cost about the same as three on the synthetic list (122 ns) and are slightly faster on the short Census surnames (75.2 ns), where the third level rarely finds anything heavy enough to split.

The frequency tables. This is the central question. On the two lists of English names the answer is that the tables add nothing measurable. Without them (prior weight 0), name sort takes 124 and 76.2 ns per name on the synthetic and Census lists; with them, the default, 118 and 81.1; with the tables alone (prior weight 10^9), 117 and 79.2. These are within about 5 % of each other, inside the run-to-run noise: the sample of $n/128$ names finds the heavy prefixes by itself, even on the synthetic list, which was drawn from the same Census data as the tables. On the multilingual list the three settings differ: 136 ns for the blend, 158 for the sample alone and 177 for the tables alone. The ASCII tables alone put the boundaries in the wrong places there. That the blend still beats the sample alone is not knowledge about names. Its any-position table gives every byte a little probability mass, which smooths the estimates for prefixes the sample saw only once or twice. A uniform prior would likely do the same. The name-frequency tables could still matter where no sample is affordable, for example for input that arrives as a stream.

Finishing and sample size. Finishing buckets of up to 256 names by introsort instead of MSD radix sort costs 127 against 118 ns per name (synthetic) and 112 against 81.1 (Census): on names, even small buckets are better sorted byte by byte. A larger sample ($n/16$) does not cut better buckets and costs 124 ns per name.

Where planning starts to pay. With the threshold removed, name sort plans at every size (dashed in Figure 2). Measured against plain MSD radix sort in the same run, it stays faster from 500,000 synthetic names on, and from 50,000 multilingual names on; on the 162,253 Census surnames it never does. Where planning starts to pay thus depends on the names: on long "Surname Firstname" strings from about half a million, on the multilingual list, whose names share long

		GCC			MSVC		
List	Names	`std::sort`	MSD radix	Name sort	`std::sort`	MSD radix	Name sort
Census surnames, shuffled	162,253	234	**61.4**	79.5	252	**73.0**	79.3
Census surnames, popularity order	162,253	212	**55.8**	74.9	230	**75.0**	77.1
Census surnames, sorted	162,253	129	**38.5**	48.5	94.4	**49.7**	53.9
Census surnames, reversed	162,253	94.6	60.0	**59.8**	97.5	65.3	**64.5**
Census surnames, upper case	162,253	235	**61.7**	79.1	251	**78.3**	80.7
Multilingual names	1,000,000	407	195	**126**	427	302	**120**

Table 4: Real name lists at full size, ns per name, median of 7; the faster of MSD radix sort and name sort in bold.

prefixes, already from tens of thousands, and on short unique surnames not at all. The library's fixed threshold of 131,072 names came from exploratory runs and fits none of the three well; a threshold derived from the sample (average name length, how many names share their first bytes) would be the better design.

7.3 Real name lists

Table 4 shows the real lists. Every list sorted correctly in both orders. On the Census surnames name sort is 1.58–2.98× faster than `std::sort` (GCC), but plain MSD radix sort is faster still on most orders. The surnames are short and unique, so MSD radix sort separates them in two or three cheap passes, and the plan's sample and trie are pure overhead at this size. Input order makes no qualitative difference: sorted, reversed and popularity-ordered lists behave like the shuffled one, and the upper-case list, which the title-case priors do not describe, does not suffer. On the multilingual list, where 79 % of the names contain non-ASCII bytes, name sort is 3.23× faster than `std::sort` and 1.55× faster than MSD radix sort (MSVC: 3.56× and 2.52×). The sample handles input the tables know nothing about.

7.4 Numeric sorts

Figure 3 and Table 5 show the numeric sorts. At 2^{24} uniform keys, radix sort is 3.48× (u32), 1.55× (u64) and 1.92× (f64) faster than `std::sort` with GCC, and 3.82, 1.88 and 2.28× with MSVC; skewed and few-unique data keep most of that. It loses on sorted input (0.72× for u64), which introsort's partitioning handles in a fraction of the random case while radix sort still moves every key; a check for sorted runs would remove that case. Bucket sort is about level with `std::sort` on random data with GCC (1.01× for f64) and somewhat faster with MSVC (1.30×), and loses on sorted and few-unique data: the per-bucket introsort is what it saves, and introsort is fast on 32 keys. The AVX2 bitonic sort beats `std::sort` on u32 keys up to 2 Mi keys, while they fit in the caches; beyond that its $O(n \log^2 n)$ passes over memory lose, as expected. The AVX2 kernel is what makes it competitive: at 2^{17} keys it is 2.65× faster than the scalar network on u32 keys and 1.55× on u64 keys, whose vectors hold half as many keys.

Bitonic hybrids and a GPU. A separate GCC run (Table 6) measures three uses of the bitonic network that the library added after the main study. *Bitonic merge sort* sorts blocks of 128 keys by the network and merges the runs with an AVX2 merge network that merges a vector of keys at a time [5], first inside the L2 cache, then across memory. On u32 keys it is 3.40× faster than `std::sort` at 2^{24} keys and 3.64× at 2^{16}; on 64-bit keys, with half the lanes per vector, 1.29×. *Bitonic bucket sort* finishes the buckets of bucket sort with the network instead of introsort. No

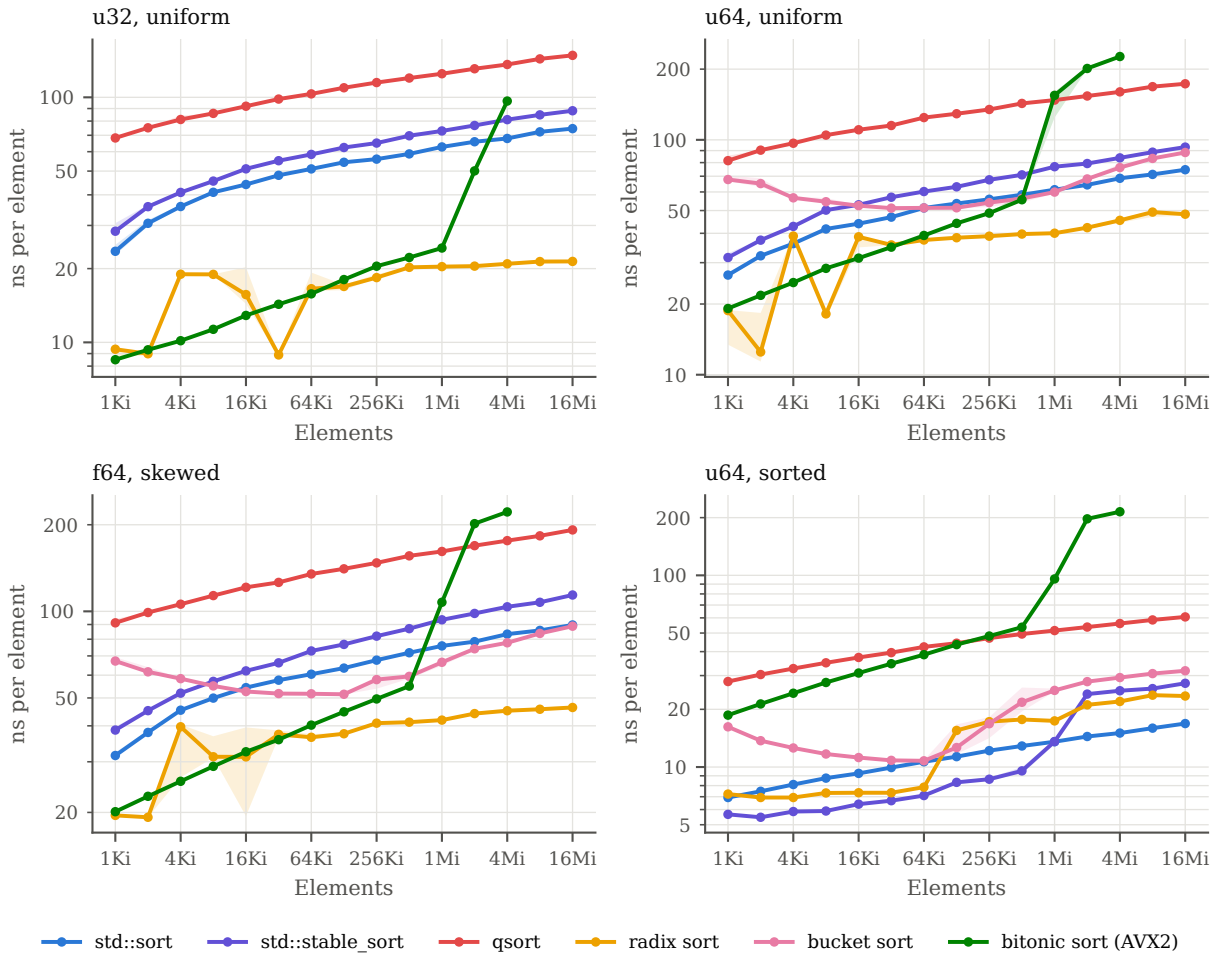


Figure 3: Numeric sorts, GCC: ns per element, median and interquartile range.

bucket size from 32 to 1,024 made it competitive (1.10× on u32 at 2^{24}): scattering into thousands of buckets costs more than the networks save. *Bitonic sort on a GPU*, an NVIDIA GeForce GTX 1660 SUPER through OpenCL, sorts blocks of 1,024 keys in local memory and finishes with global and local merge stages. How fast it is depends on how often it runs. Interleaved with the CPU sorts, as in Table 6, the GPU idles for up to a second between its runs and lowers its clocks; runs of a few milliseconds then finish before the clocks are back up, and at sizes between about 2^{20} and 2^{22} keys its median is several times worse than its best runs. Even so it is 5.15× faster than `std::sort` on 2^{24} u32 keys. Run back to back, the GPU's times are steady: 5.49 ns per key at 2^{24} u32 keys (13.8× faster than `std::sort`) and 10.6 ns at 2^{24} u64 keys (6.80×), copies included, and it is faster than every CPU sort in the study from 32 Ki u32 keys and 16 Ki u64 keys on. The library sends bitonic sorts to the GPU by default from 65,536 keys on, a margin for a GPU that has to clock up first.

8 Threats to validity

All measurements come from one machine; processors with a larger last-level cache or more memory bandwidth will move the points where planning, radix sort and bitonic sort start or stop paying. The synthetic names are drawn from the data the priors were built from, which favours the priors;

Type	Data	GCC			MSVC		
		`std::sort` ns	Radix speed-up	Bucket speed-up	`std::sort` ns	Radix speed-up	Bucket speed-up
u32	uniform	74.5	3.48×	–	87.7	3.82×	–
	skewed	59.8	2.94×	–	65.5	3.03×	–
	sorted	11.8	0.57×	–	12.4	0.60×	–
	few-unique	25.0	2.70×	–	13.6	1.42×	–
u64	uniform	74.6	1.55×	0.84×	89.6	1.88×	1.09×
	skewed	59.3	2.53×	0.81×	65.9	2.73×	0.94×
	sorted	16.9	0.72×	0.53×	14.9	0.63×	0.29×
	few-unique	24.7	2.39×	0.44×	13.7	1.29×	0.23×
f64	uniform	89.8	1.92×	1.01×	109	2.28×	1.30×
	skewed	89.7	1.94×	1.01×	108	2.33×	1.29×
	sorted	15.8	0.50×	0.47×	12.0	0.38×	0.23×
	few-unique	28.6	1.87×	0.48×	16.7	1.07×	0.28×

Table 5: Numeric sorts at 2^{24} keys: `std::sort` in ns per element, radix and bucket sort as speed-up over it (above 1: faster).

the ablation shows that even so they do not matter measurably. The Census list holds each surname once, while a real register repeats common names, which favours MSD radix sort less than our Census list does and the plan more. We had no real list of people from outside the United States; the multilingual list consists of words, not names. The baselines are one implementation each; a tuned string sort such as the one of Kärkkäinen and Rantala [10] would be a stronger baseline than our plain MSD radix sort, and would likely narrow name sort's lead on large inputs. The GPU also drives the machine's display, and its times are the least stable in this study: between repetitions of the same configuration they can differ several-fold, which we attribute to the display and to the GPU's power states. We report medians, and the copies to and from the device are always included; a GPU that only computes, or data that stays on the device, would be faster. Finally, all CPU times are single-threaded.

9 Conclusion

Person names are unevenly distributed, and a bucket sort that cuts its buckets from the distribution of their first three letters is fast: about 3× faster than `std::sort` on a million names, and faster than a plain MSD radix sort on long or non-English names. But the evaluation attributes that speed to the bucket structure and the MSD radix sort that finishes it, not to knowing the distribution in advance: a small sample of the input finds the same boundaries as Census frequency tables, deeper and more even plans are slower, and on short, unique surnames a plain MSD radix sort is better. A distribution model fitted outside the input, the idea that motivated name sort, did not pay on this hardware.

The numeric sorts confirm that the standard libraries leave speed on the table for large arrays of plain keys: LSD radix sort, the AVX2 bitonic merge sort and, where a GPU is available, a bitonic sort on the GPU are several times faster than introsort. All of it, including the negative results, is reproducible from the published library, data and scripts.

		Radix			Bitonic merge			Bitonic bucket			Bitonic, GPU		
Type	Data	64 Ki	1 Mi	16 Mi	64 Ki	1 Mi	16 Mi	64 Ki	1 Mi	16 Mi	64 Ki	1 Mi	16 Mi
u32	uniform	5.41	3.06	3.48	3.64	3.48	3.40	1.18	1.33	1.10	6.41	14.0	5.15
	skewed	3.30	2.88	2.93	3.63	3.16	2.71	1.18	1.06	0.88	6.72	2.31	3.16
	sorted	2.07	0.58	0.57	0.58	0.53	0.52	0.42	0.40	0.36	1.34	2.26	1.63
	few-unique	3.29	2.62	2.73	1.52	1.30	1.14	0.60	0.51	0.46	2.87	2.47	2.30
u64	uniform	1.56	1.52	1.57	1.46	1.33	1.29	0.96	0.99	0.94	4.36	0.55	2.47
	skewed	2.23	2.39	2.49	1.43	1.22	1.04	0.94	0.87	0.74	5.77	1.03	3.06
	sorted	1.16	0.15	0.63	0.27	0.25	0.27	0.37	0.30	0.37	1.25	0.11	0.76
	few-unique	2.74	2.13	2.22	0.58	0.48	0.41	0.55	0.47	0.42	2.23	0.51	1.17
f64	uniform	1.66	1.79	1.93	1.71	1.58	1.54	1.13	1.21	1.12	6.72	1.34	3.86
	skewed	1.59	1.80	1.94	1.68	1.60	1.53	1.12	1.26	1.12	5.66	1.33	3.59
	sorted	0.50	0.32	0.50	0.29	0.27	0.27	0.34	0.33	0.37	1.02	0.29	0.83
	few-unique	1.70	1.79	1.86	0.69	0.57	0.49	0.64	0.58	0.49	2.47	0.59	1.31

Table 6: Bitonic hybrids and the GPU (one GCC run; all results all correct): speed-up over `std::sort` in the same run. The GPU times include the copies to and from the device.

Availability

The library (C ABI and a C++17 layer), the table generator, the benchmark harness, the raw results and the scripts that produce every figure, table and number in this paper are at `https://github.com/attilatorda/SpeedUp` (AGPL-3.0):

- sorts: `src/algorithms/sort/`, API `include/speedup/algorithms/sort.h`;
- tables: `codegen/name_tables/build_name_tables.py`;
- harness: `benchmarks/sort/bench_sort.cpp`;
- runner: `benchmarks/scripts/run_sort_benchmarks.py`;
- results: `benchmarks/outputs/sort_*.csv` and `namesort_*.csv`;
- figures, tables and numbers: `benchmarks/scripts/sorting_paper_numbers.py`.

References

[1] Michael Axtmann, Sascha Witt, Daniel Ferizovic, and Peter Sanders. In-place parallel super scalar samplesort (IPS^{4}o). In *25th Annual European Symposium on Algorithms (ESA 2017)*, volume 87 of *LIPIcs*, pages 9:1–9:14, 2017. doi: 10.4230/LIPIcs.ESA.2017.9.

[2] Kenneth E. Batcher. Sorting networks and their applications. In *Proceedings of the AFIPS Spring Joint Computer Conference*, volume 32, pages 307–314, 1968.

[3] Jon L. Bentley and Robert Sedgewick. Fast algorithms for sorting and searching strings. In *Proceedings of the 8th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 360–369, 1997.

[4] Lukas Bergdoll and Orson R. L. Peters. driftsort and ipnsort: the sort implementations of the Rust standard library. `https://github.com/Voultapher/driftsort`, 2024.

[5] Jatin Chhugani, Anthony D. Nguyen, Victor W. Lee, William Macy, Mostafa Hagog, Yen-Kuang Chen, Akram Baransi, Sanjeev Kumar, and Pradeep Dubey. Efficient implementation of sorting on multi-core SIMD CPU architecture. *Proceedings of the VLDB Endowment*, 1(2):1313–1324, 2008.

[6] Mark Davis, Ken Whistler, and Markus Scherer. Unicode technical standard #10: Unicode collation algorithm. `https://www.unicode.org/reports/tr10/`. The Unicode Consortium.

[7] W. D. Frazer and A. C. McKellar. Samplesort: A sampling approach to minimal storage tree sorting. *Journal of the ACM*, 17(3):496–507, 1970. doi: 10.1145/321592.321600.

[8] Torsten Hoefler and Roberto Belli. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC15)*, pages 73:1–73:12, 2015. doi: 10.1145/2807591.2807644.

[9] IEEE. IEEE standard for floating-point arithmetic. IEEE Std 754-2019, 2019.

[10] Juha Kärkkäinen and Tommi Rantala. Engineering radix sort for strings. In *String Processing and Information Retrieval (SPIRE 2008)*, volume 5280 of *Lecture Notes in Computer Science*, pages 3–14. Springer, 2008.

[11] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, 2nd edition, 1998.

[12] Ani Kristo, Kapil Vaidya, Ugur Çetintemel, Sanchit Misra, and Tim Kraska. The case for a learned sorting algorithm. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 1001–1016, 2020. doi: 10.1145/3318464.3389752.

[13] Peter M. McIlroy, Keith Bostic, and M. Douglas McIlroy. Engineering radix sort. *Computing Systems*, 6(1):5–27, 1993.

[14] Duane Merrill and Andrew Grimshaw. High performance and scalable radix sorting: A case study of implementing dynamic parallelism for GPU computing. *Parallel Processing Letters*, 21 (2):245–272, 2011.

[15] Microsoft. Array.sort method (.NET API documentation). `https://learn.microsoft.com/dotnet/api/system.array.sort`. Accessed October 2026.

[16] J. Ian Munro and Sebastian Wild. Nearly-optimal mergesorts: Fast, practical sorting methods that optimally adapt to existing runs. In *26th Annual European Symposium on Algorithms (ESA 2018)*, volume 112 of *LIPIcs*, pages 63:1–63:16, 2018.

[17] David R. Musser. Introspective sorting and selection algorithms. *Software: Practice and Experience*, 27(8):983–993, 1997.

[18] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong! In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 265–276, 2009. doi: 10.1145/1508244.1508275.

[19] OpenJDK. JDK-8266431: Dual-pivot quicksort improvements (radix sort). `https://bugs.openjdk.org/browse/JDK-8266431`. Accessed October 2026.

[20] Oracle. java.util.arrays (Java SE 21 API specification). https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Arrays.html, 2023.

[21] Orson R. L. Peters. Pattern-defeating quicksort. arXiv:2106.05123, 2021.

[22] Tim Peters. listsort.txt: the CPython list sort (timsort). https://github.com/python/cpython/blob/main/Objects/listsort.txt, 2002.

[23] Steven J. Ross, Francisco José Tapia, and Orson R. L. Peters. Boost.Sort. https://www.boost.org/doc/libs/release/libs/sort/. Accessed October 2026.

[24] Peter Sanders and Sebastian Winkel. Super scalar sample sort. In *Algorithms – ESA 2004*, volume 3221 of *Lecture Notes in Computer Science*, pages 784–796. Springer, 2004.

[25] Nadathur Satish, Mark Harris, and Michael Garland. Designing efficient sorting algorithms for manycore GPUs. In *Proceedings of the IEEE International Symposium on Parallel and Distributed Processing (IPDPS)*, 2009.

[26] Ranjan Sinha and Justin Zobel. Cache-conscious sorting of large sets of strings with dynamic tries. *ACM Journal of Experimental Algorithmics*, 9, 2004.

[27] The Go Authors. Go 1.19 release notes: package sort uses pattern-defeating quicksort. https://go.dev/doc/go1.19, 2022.

[28] U.S. Census Bureau. Frequently occurring first names and surnames from the 1990 census. https://www2.census.gov/topics/genealogy/1990surnames/, 1995. Accessed October 2026.

[29] U.S. Census Bureau. Frequently occurring surnames from the 2010 census. https://www2.census.gov/topics/genealogy/2010surnames/names.zip, 2016. Accessed October 2026.

[30] Simon Zünd. Getting things sorted in V8. https://v8.dev/blog/array-sort, 2018.